

ESCUELA DE INGENIERÍA INFORMÁTICA DE OVIEDO

Arquitectura de Computadores

Curso 2025-2026

Trabajo en grupo

Díaz Mendaña, Diego - UO301887

García Pernas, Pablo - UO300167

Rama García, Mateo - UO300710

Suárez Fernández, Fernando - UO300028

Grupo de prácticas: PL.1 - B

Titulación: PCEO Informática y Matemáticas

28 de noviembre de 2025

Índice

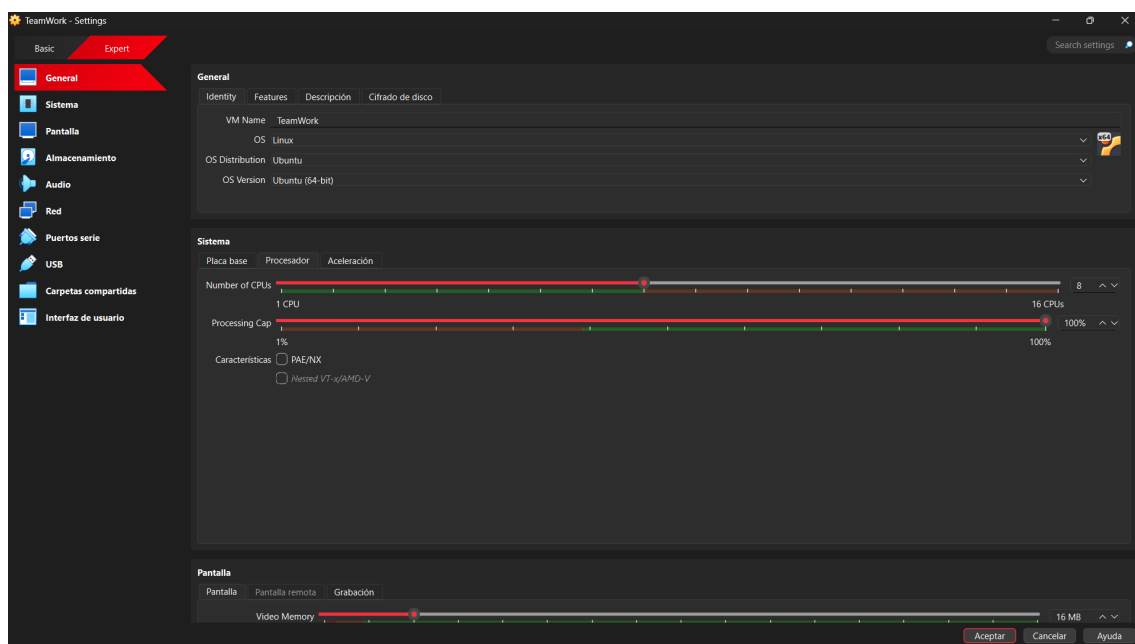
1. Introducción	2
2. Máquina virtual	2
3. Desarrollo del algoritmo	2
3.1. Implementación del algoritmo	3
4. Desarrollo del Proyecto	4
4.1. Fase 1: Implementación Monohilo	4
4.2. Fase 2: Optimización SIMD y Multihilo	4
4.2.1. Optimización SIMD	4
4.2.2. Paralelización Multihilo	6
5. Resultados y Análisis	8
5.1. Medición del rendimiento	8
5.2. Análisis de la versión multihilo	9
5.3. Análisis de la versión SIMD	9
5.4. diffImages	10
6. Metodología de trabajo	11
6.1. Distribución de tareas	11

1. Introducción

La presente memoria recoge el desarrollo del proyecto correspondiente a la asignatura *Arquitectura de Computadores*, impartida durante el curso académico 2025-2026 en la Escuela de Ingeniería Informática de Oviedo. El objetivo del proyecto es aplicar los conocimientos adquiridos en la asignatura para diseñar e implementar un filtro de imágenes en lenguaje C/C++. Asimismo, se realiza un análisis del rendimiento del filtro y una comparación de la versión monohilo con versiones optimizadas mediante técnicas SIMD y multihilo.

2. Máquina virtual

Para el desarrollo del proyecto se utilizó la máquina virtual proporcionada por los profesores, a continuación se muestra una captura de pantalla con la configuración del sistema:



Además, como se ha indicado en el enunciado del trabajo se adjunta archivo `cpuinfo` con las características de la CPU usada para el análisis del rendimiento.

3. Desarrollo del algoritmo

Se ha desarrollado el algoritmo de inversión en blanco y negro (*Algoritmo número #3*) empleando la plantilla facilitada por el profesorado.

El algoritmo consiste en calcular el valor de luminancia de cada pixel, para ello se sigue la fórmula de luminancia del espacio de color *UIT-R BT.601-7* con los pesos aproximados:

$$L_i = 0.3 \cdot R_i + 0.59 \cdot G_i + 0.11 \cdot B_i \quad \text{con} \quad \begin{cases} L_i \text{ valor luminancia pixel } i \\ R_i \text{ componente roja pixel } i \\ G_i \text{ componente verde pixel } i \\ B_i \text{ componente azul pixel } i \end{cases}$$

Una vez obtenido este valor se invierte respecto al brillo máximo para obtener el efecto de inversión en blanco y negro.

$$L'_i = 255 - L_i$$

Notar que el valor obtenido no va a ser nunca negativo o superior a 255, es por ello que no hace falta realizar estas comprobaciones.

3.1. Implementación del algoritmo

Como la lógica del algoritmo es la misma para las 3 versiones, a continuación solo se muestra la implementación de la versión monohilo para mayor claridad, el resto se encuentra en el código fuente y se ha implementado usando el tipo `double`. Para cada pixel de la imagen se realiza el algoritmo detallado anteriormente y se guarda el resultado en los 3 canales de color del píxel de la imagen resultante.

```

1 void filter (filter_args_t args) {
2     for (uint i = 0; i < args.pixelCount; i++) {
3         data_t L = *(args.pRsrc + i) * R_WEIGHT +
4                     *(args.pGsrc + i) * G_WEIGHT +
5                     *(args.pBsrc + i) * B_WEIGHT;
6
7         L = MAX_BRIGHTNESS - L;
8
9         *(args.pRdst + i) = L;
10        *(args.pGdst + i) = L;
11        *(args.pBdst + i) = L;
12    }
13 }
```

Dado que el tiempo de ejecución debía situarse entre 8 y 15 segundos en la versión monohilo, se realizó un bucle de 250 iteraciones de la función `filter` para mantener el tiempo dentro del rango esperado:

```

1 for (int i = 0; i < N_ITERATIONS; i++) {
2     filter(filter_args);
3 }
```

Para mantener la coherencia en las mediciones, se ha decidido mantener este enfoque en todas las implementaciones del algoritmo.

4. Desarrollo del Proyecto

4.1. Fase 1: Implementación Monohilo

En la primera fase del proyecto se desarrolló la versión monohilo del algoritmo, centrada en la comprensión del flujo de procesamiento y en la verificación de la corrección del filtro aplicado sobre las imágenes. El proceso es el descrito en la [sección anterior](#). A continuación, se muestran la imagen original y la imagen resultante tras aplicar el filtro:



Figura 1: Imagen original



Figura 2: Imagen resultante (monohilo)

4.2. Fase 2: Optimización SIMD y Multihilo

En la segunda fase del proyecto, se optimizó el algoritmo mediante el uso de instrucciones SIMD y la paralelización con múltiples hilos. No obstante, los resultados no han mostrado una mejora significativa en el rendimiento, que posteriormente se analizará en detalle.

4.2.1. Optimización SIMD

Para la optimización SIMD, se ha empleado la librería `immintrin.h` que proporciona acceso a las instrucciones SSE y AVX. Se han empleado paquetes de 128 bits (`__m128d`) para procesar dos valores `double` simultáneamente. El objetivo es aprovechar el paralelismo de datos a nivel de instrucción para reducir el número de iteraciones necesarias procesando múltiples píxeles en cada ciclo.

Para la inicialización de los vectores, las constantes de algoritmo se cargaron replicando el valor en ambos elementos del vector:

```
1  typedef __m128d simd_t;  
2  
3  const simd_t r_weight = _mm_set1_pd(R_WEIGHT);  
4  const simd_t g_weight = _mm_set1_pd(G_WEIGHT);  
5  const simd_t b_weight = _mm_set1_pd(B_WEIGHT);  
6  const simd_t max_brightness = _mm_set1_pd(MAX_BRIGHTNESS);
```

Para tener un acceso eficiente a los datos, se utiliza `_mm_malloc()` con alineamiento a 16 bytes (tamaño de `__m128d`). Además, para garantizar que cada componente de color ocupe un número de posiciones múltiplo del tamaño del paquete SIMD, se calcula la variable `pixelCountAligned`. Esta variable almacena el número de píxeles ajustado al siguiente múltiplo `ITEMS_PER_PACKET` (2 en nuestro caso).

Si no se realizase este proceso, al almacenarse de manera contigua las componentes de color en memoria podría haber un paquete que tuviese componentes de distinto color. El padding añadido se elimina una vez se ha realizado el algoritmo y antes de la creación de la imagen final.

```
1  uint pixelCountAligned = ((filter_args.pixelCount + ITEMS_PER_PACKET - 1) /
    ITEMS_PER_PACKET) * ITEMS_PER_PACKET;
2  pDstImage = (data_t *) _mm_malloc (pixelCountAligned * nComp *
    sizeof(data_t), 16);
```

El bucle principal del filtro procesa la imagen en paquetes de dos píxeles. Para cada paquete, se cargan los valores RGB mediante `_mm_loadu_pd()`. Aunque los datos están alineados en memoria y se podría emplear `_mm_load_pd()` para mejorar levemente la eficiencia, se optó por la versión no alineada para evitar posibles errores si las direcciones de memoria no cumplen estrictamente el alineamiento requerido. Posteriormente, se calcula la luminosidad ponderada empleando `_mm_mul_pd()` y `_mm_add_pd()`, y finalmente se invierte con `_mm_sub_pd()`.

Hemos decidido no utilizar la instrucción `_mm_fmadd_pd()` ya que esta redondea una única vez, mientras que multiplicar y sumar separadamente redondea dos veces, lo que puede introducir pequeñas diferencias respecto a la imagen monohilo.

```
1  const uint nPackets = (args.pixelCount / ITEMS_PER_PACKET);
2  for(uint i = 0; i < nPackets; i++){
3      const uint pos = i * ITEMS_PER_PACKET;
4      const simd_t vr = _mm_loadu_pd(args.pRsrc + pos);
5      const simd_t vg = _mm_loadu_pd(args.pGsrc + pos);
6      const simd_t vb = _mm_loadu_pd(args.pBsrc + pos);
7
8      simd_t L = _mm_mul_pd(vr, r_weight);
9      L = _mm_add_pd(L, _mm_mul_pd(g_weight,vg));
10     L = _mm_add_pd(L, _mm_mul_pd(b_weight,vb));
11     L = _mm_sub_pd(max_brightness, L);
12
13     *(simd_t*)(args.pRdst + pos) = L;
14     *(simd_t*)(args.pGdst + pos) = L;
15     *(simd_t*)(args.pBdst + pos) = L;
16 }
```

Dado que el número total de píxeles puede no ser múltiplo exacto de 2, hay un bucle residual que procesa los píxeles restantes de forma escalar, empleando la misma lógica que en la versión monohilo. Finalmente, se libera la memoria alineada con `_mm_free()` para evitar fugas de memoria. A continuación, se muestran las imágenes resultantes:



4.2.2. Paralelización Multihilo

Para la paralelización multihilo, se utilizó la librería `pthread.h` para crear y gestionar múltiples hilos de ejecución. El objetivo es dividir la carga de trabajo entre varios núcleos del procesador, empleando 8 hilos de ejecución (`NUM_THREADS = 8`) para distribuir la imagen en fragmentos que se pueden procesar de forma concurrente.

La estructura `filter_args_t` se amplió con dos campos adicionales (`start` y `end`) que delimitan el rango de píxeles que cada hilo debe procesar, para trabajar sobre porciones independientes de la imagen:

```
1  typedef struct {
2      data_t *pRsrc;
3      data_t *pGsrc;
4      data_t *pBsrc;
5      data_t *pRdst;
6      data_t *pGdst;
7      data_t *pBdst;
8      uint pixelCount;
9      uint start, end;
10 } filter_args_t;
```

La función `thread_filter()` se ejecuta en cada hilo, donde recibe los punteros a los datos y rango de píxeles a procesar. Primero ajusta los punteros aplicando el *offset start*, calcula la longitud del segmento y procesa los píxeles dentro de ese rango aplicando el mismo algoritmo que en la versión monohilo:

```
1  void* thread_filter(void* args){
2      filter_args_t* tArgs = (filter_args_t*) args;
3
4      data_t *pR = tArgs->pRsrc + tArgs->start;
5      data_t *pG = tArgs->pGsrc + tArgs->start;
6      data_t *pB = tArgs->pBsrc + tArgs->start;
7      data_t *pRdst = tArgs->pRdst + tArgs->start;
8      data_t *pGdst = tArgs->pGdst + tArgs->start;
9      data_t *pBdst = tArgs->pBdst + tArgs->start;
10
11      uint length = tArgs->end - tArgs->start;
```

```

12
13     // Procesamiento análogo al filtro monohilo
14     pthread_exit(NULL);
15 }

```

La función `execute_multithreaded_filter()` se encarga de coordinar la ejecución paralela. Para ello, calcula los píxeles por hilo, crea los hilos con `pthread_create()` y espera a que terminen con `pthread_join()`. El último hilo se encarga de procesar cualquier píxel restante en caso de que el total no sea divisible por el número de hilos:

```

1 void execute_multithreaded_filter(filter_args_t* filter_args, uint
  pixelsPerThread) {
2     pthread_t threads[NUM_THREADS];
3     filter_args_t thread_args[NUM_THREADS];
4
5     for (uint t = 0; t < NUM_THREADS; t++) {
6         thread_args[t] = *filter_args;
7         thread_args[t].start = t * pixelsPerThread;
8         thread_args[t].end = (t == NUM_THREADS - 1) ?
  filter_args->pixelCount : (t + 1) * pixelsPerThread;
9
10        if(pthread_create(&threads[t], NULL, thread_filter,
  &thread_args[t]) != 0){
11            fprintf(stderr, "Error creating thread");
12            exit(EXIT_FAILURE);
13        }
14    }
15
16    for (uint t = 0; t < NUM_THREADS; t++) {
17        if (pthread_join(threads[t], NULL) != 0) {
18            fprintf(stderr, "Error joining thread %d\n", t);
19            exit(EXIT_FAILURE);
20        }
21    }
22 }

```

Finalmente, en la función principal, se calcula el número de píxeles por hilo y se llama a `execute_multithreaded_filter()` para iniciar el procesamiento paralelo. Esta función se ejecutará el número de veces fijado para medir el tiempo total de ejecución. A continuación, se muestran las imágenes resultantes:



5. Resultados y Análisis

5.1. Medición del rendimiento

Para la obtención de resultados, se midió el tiempo de ejecución de las distintas versiones del filtro (monohilo, SIMD y multihilo) utilizando la función `clock_gettime()`. Para evitar variaciones debidas a la carga del sistema, se realizaron múltiples ejecuciones (10 repeticiones) y se calculó el tiempo promedio. A continuación se muestran los resultados obtenidos:

Versión	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10
Monohilo	10.27	10.54	10.64	10.43	10.49	10.30	10.33	10.38	10.35	10.21
Multihilo	6.72	6.58	6.57	6.60	6.54	6.49	6.47	6.63	6.56	6.72
SIMD	13.96	14.22	13.98	14.13	13.70	13.82	14.12	13.71	14.10	14.29

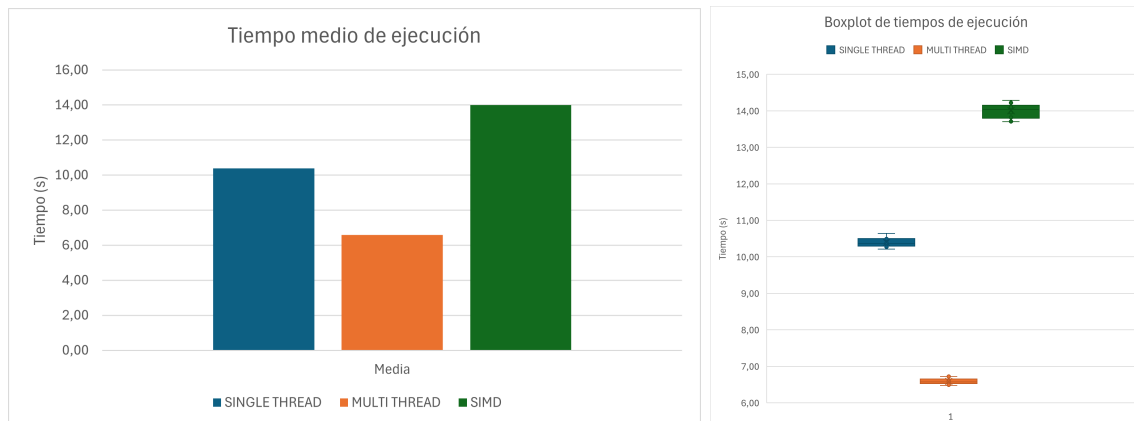
A partir de los datos obtenidos, se calcularon una serie de medidas estadísticas para resumir la información, aplicando medidas de centralización y dispersión. Cabe destacar que, por la presencia de algunos *outliers*, la aproximación del intervalo de confianza mediante el *Teorema Central del Límite* ha dado valores algo imprecisos, por lo que se ha añadido un calculo más preciso empleando la distribución *t-Student* (identificado como *IC - TS*).

Versión	Media (s)	DT (s)	IC - TCL (s)	IC - TS (s)	Product. ($\frac{\text{tarear}}{\text{min}}$)
Monohilo	10.39	0.13	[10.13, 10.66]	[10.27, 10.51]	5.77
Multihilo	6.59	0.08	[6.42, 6.76]	[6.51, 6.67]	9.11
SIMD	14.00	0.21	[13.59, 14.42]	[13.81, 14.19]	4.29

A continuación se han calculado las respectivas aceleraciones de las versiones optimizadas respecto a la versión monohilo:

$$\text{Aceleración Multihilo} = \frac{10.39}{6.59} \approx 1.58 \quad \text{Aceleración SIMD} = \frac{10.39}{14.00} \approx 0.74$$

A modo de resumen visual, se presentan los siguientes gráficos que ilustran los tiempos de ejecución y las aceleraciones obtenidas:



5.2. Análisis de la versión multihilo

La implementación multihilo ha mostrado una mejora de rendimiento de aproximadamente **1.58** respecto a la versión monohilo. Aunque esta mejora es significativa, está muy lejos del teórico ideal de $\times 16$ (usando 16 hilos). No obstante se han utilizado 8 hilos por motivos que se tratarán más adelante. Todo esto resulta en una eficiencia del **19.75 %**, lo cual indica que existen cuellos de botella en algunos puntos del proceso. Las posibles causas incluyen:

- **Contención de recursos:** Aunque cada hilo trabaja sobre un segmento independiente de la imagen, a partir de cierto número de hilos aparece contención en la memoria caché. Cuando demasiados hilos acceden simultáneamente a regiones de memoria cercanas aumenta el número de fallos de caché. Esto hace que el rendimiento pase a estar limitado por el subsistema de memoria, es por ello que hemos decidido limitar el número de hilos a 8.
- **Overhead de creación/destrucción de hilos:** En cada iteración se crean y destruyen 8 hilos, lo que introduce un coste adicional en el tiempo total. Se ha mantenido este enfoque por indicaciones presentes en la rúbrica de evaluación.
- **Tamaño de trabajo insuficiente por hilo:** Dado que la imagen no es extremadamente grande, las operaciones de sincronización pueden suponer un tiempo considerable en comparación con las operaciones de suma/multiplicación realizadas.

5.3. Análisis de la versión SIMD

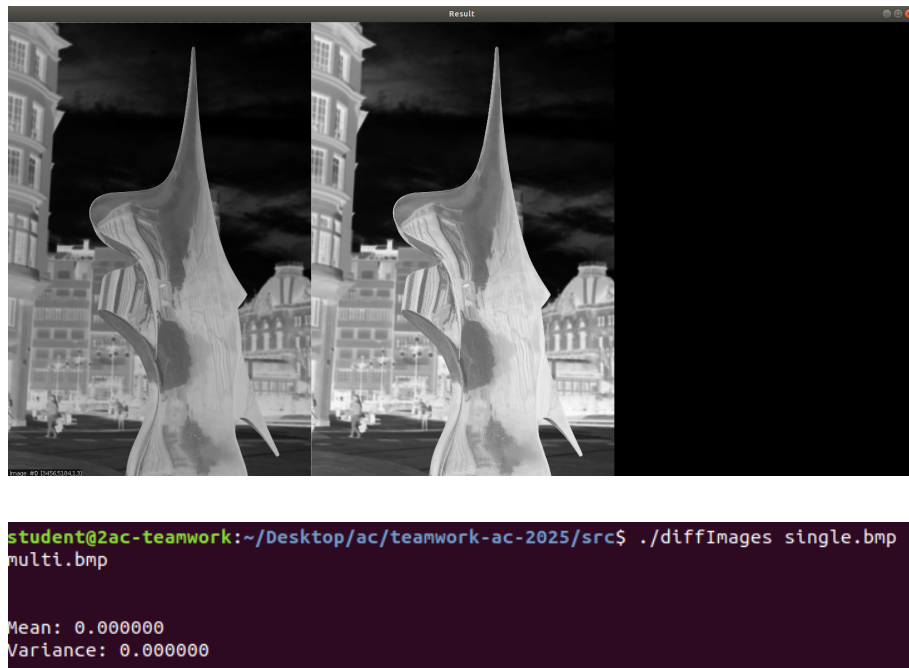
Los resultados de la versión SIMD son especialmente sorprendentes: en lugar de mejorar el rendimiento, esta implementación resulta **0.74x más lenta** que la versión monohilo original. Se ha comprobado en varios ordenadores con diferentes procesadores y, en todos, los resultados son similares. Se ha verificado que todos ellos tuvieran soporte para las instrucciones necesarias y que el código estuviera optimizado. Algunas posibles causas de este comportamiento incluyen:

- **Overhead en la gestión de memoria:** La utilización de memoria alineada y las operaciones de carga/almacenamiento SIMD pueden introducir un overhead que no compensa la paralelización a nivel de datos, especialmente si el tamaño de los datos no es lo suficientemente grande.
- **Limitaciones del compilador:** Es posible que el compilador no esté optimizando adecuadamente el código SIMD, o que las instrucciones generadas no sean las más eficientes para el hardware específico.

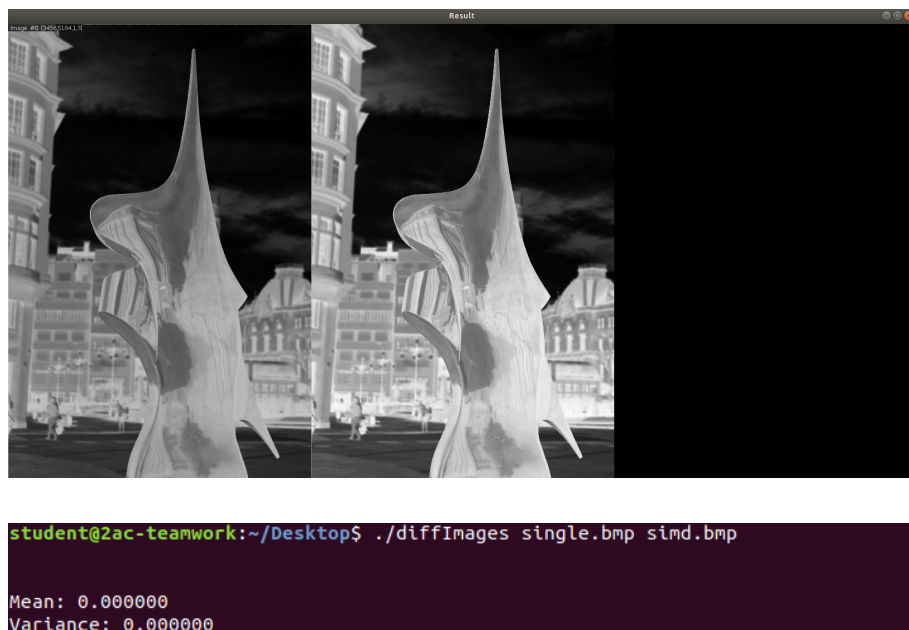
5.4. diffImages

A continuación, se muestra el resultado de aplicar la herramienta `diffImages` para comparar las imágenes generadas por las distintas versiones multihilo y SIMD del filtro con la imagen generada por la versión monohilo. En ambos casos, el resultado es una imagen completamente negra, lo que indica que no hay diferencias entre las imágenes. Además, se muestran las métricas proporcionadas por `diffImages`.

■ Multihilo



■ SIMD



6. Metodología de trabajo

El desarrollo del proyecto se llevó a cabo mediante el uso riguroso de *Git* y *GitHub*. El flujo de trabajo adoptado se estructuró conforme al modelo de *feature branching*. Cada funcionalidad fue desarrollada en una rama independiente derivada de la rama principal `main`, y su incorporación al proyecto se realizó mediante *pull requests*. Este procedimiento garantizó un proceso de revisión de código sistemático, facilitando la detección de inconsistencias y la resolución de conflicto. El repositorio que recoge el trabajo desarrollado se encuentra disponible en *GitHub*: <https://github.com/Mendana/teamwork-ac-2025>.

6.1. Distribución de tareas

La distribución del trabajo se realizó asignando a cada miembro del equipo un conjunto definido de responsabilidades, correspondientes a distintas etapas del proyecto. Tras la fase de implementación individual, se desarrolló un proceso de revisión conjunta, con el fin de garantizar una visión compartida del funcionamiento global del trabajo. Las optimizaciones del código se realizaron de forma cruzada para que todos los integrantes trabajasen sobre el código. A continuación se detalla la contribución específica de cada miembro del equipo:

- **Fernando Suárez Fernández:** Desarrollo del algoritmo de filtro de la versión monohilo y estructura del SIMD. Revisión y corrección de errores tanto en el código como en la memoria.
- **Pablo García Pernas:** Estructura de la versión monohilo, optimización del SIMD y testing de los algoritmos. Revisión y corrección de errores tanto en el código como en la memoria.
- **Mateo Rama García:** Implementación de la versión multihilo y redacción de parte de la memoria. Optimización de la versión SIMD, testing de algoritmos y corrección de errores en el código.
- **Diego Díaz Mendaña:** Organización del repositorio y asignación de tareas. Optimización de la versión multihilo y análisis de resultados obtenidos. Redacción de parte de la memoria y revisión de errores en el código.

Estimamos que la distribución del trabajo ha sido equitativa (25%), permitiendo a cada miembro del equipo contribuir de manera significativa al desarrollo del proyecto.